

实验七：贪心法

对应课程目标 3

班级：7 班

姓名：卢瑶

学号：20211018341

一、实验源代码、伪代码、结果和复杂度分析

第 1 关:找零钱

设计一个贪婪算法，使得找的钱币张数最少。商店售货员找给 1 个顾客 元，用以下七种面值的纸币:100 元，50 元，20 元，10 元，5 元，2 元，1 元。思考:如果商店售货员找给 1 个顾客 140 元，假设钱币的面值有九种:100 元，70 元，50 元，20 元，10 元，7 元，5 元，2 元，1 元。用贪婪算法得到的是该问题的最优解吗？

伪码描述:

```
int main() {  
  
    for(int i = 0; i < n; i++) {  
  
        cnt = W / money[i];  
  
        W -= cet * money;  
  
        printf("%d 元 需要用 %d 张\n", money[i], cnt);  
  
        if(W == 0)  
  
            break;  
  
    }  
}
```

```
}
```

源代码:

```
#include <stdio.h>
// #include <iostream>
// using namespace std;
int money[] = {100, 50, 20, 10, 5, 2, 1};
int main()
{
    /***** Begin *****/
    int W;
    scanf("%d", &W);
    int cnt = 0;
    for(int i = 0; i < sizeof(money); i++){ //打印 1 - n 行
        cnt = W / money[i];
        W -= cnt * money[i];
        printf("%d 元 %d 张\n", money[i], cnt);
        if(W == 0)
            break;
    }
    return 0;
    /***** End *****/
}
```

运行结果:

```
123
100元 1张
50元 0张
20元 1张
10元 0张
5元 0张
2元 1张
1元 1张
```

时间复杂度分析:

$O(n)$

空间复杂度分析:

$O(n)$

第 2 关:求一个数列的极差。

将 n 个正整数作成的一个数列,进行如下操作:每一次删除其中的两个数 a 和 b ,然后在数列中加入一个数 $a+b$,如此下去直至数列中剩下一个数。在所有按这种操作方式最后得到的数中,最大的记作 $\max$,最小的记作 $\min$,则该数列的极差定义为 $M=\max-\min$,请你使用贪心算法设计编程输出它们的极差。

伪码描述:

```
int main() {
```

创建数组, numbers 数组用于求最大值, copy 数组用于求最小值。

```
for(int i = 1; i < n; i++){
```

```
    sort(numbers, numbers + n);
```

```
    tmp = numbers[i] * numbers[i - 1] + 1;
```

```
    numbers[i] = tmp;
```

```
}
```

一轮下来,最后一个元素就是 $\max$ 并记录。

```
for(int i = n - 1; i >= 1; --i){
```

注意此处不用排序,从右至左总是递增。

```

        tmp = copy[i] * copy[i - 1] + 1;

        numbers[i - 1] = tmp;

    }

```

一轮下来，第一个元素就是 min 并记录。

输出极差 $\max - \min$ 即可。

```

}

```

源代码：

```

#include <iostream>
#include <stdio.h>
#include <algorithm>

using namespace std;

int Max(int n, int copy[]) {
    int tmp = 1;
    // printf("%d\n", tmp);
    for(int i=1; i < n; i++){
        sort(copy, copy + n);
        tmp = copy[i] * copy[i-1] + 1;
        copy[i] = tmp;
    // printf("%d ", copy[i]);
    }
    printf("\n");

    return copy[n - 1];
}

int Min(int n, int copy[]) {
    int tmp = 1;
    // printf("%d\n", tmp);
    for(int i=n-1; i >= 1; i--){
    //
        sort(copy, copy + n);
        tmp = copy[i] * copy[i-1] + 1;
        copy[i - 1] = tmp;
    }
}

```

```

//          printf("%d ", copy[i - 1]);
    }
    printf("\n");

    return copy[0];
}
/*
7      //输入 7 (n) 个整数
3 5 7 9 11 13 15
Max=max-min=2221298-2038489=182809
*/
int main(int argc, char *argv[]) {
    //数据接收
    int n;
    scanf("%d", &n);
    int a[n], copy[n];
    for (int i=0; i < n; i++){
        scanf("%d",&a[i]);
        copy[i] = a[i];
    }
    int max = Max(n, copy);
    int min = Min(n, a);
    printf("Max=max-min=%d-%d=%d\n", max, min, max-min);
}

```

运行结果:

7

3 5 7 9 11 13 15

Max=max-min=2221298-2038489=182809

时间复杂度分析:

$O(n^2 \log n)$

空间复杂度分析:

$O(n)$

第 3 关:埃及分数。

埃及分数是分子为 1 的分数，每个真分数可以表示成若干埃及分数之和。现给定 1 个真分数，求最少的埃及分数个数。

伪码描述:

```
int main() {  
  
    接收 a, b;  
  
    if(a >= b)  
        返回错误，不是真分数;  
  
    else if(a == 1 || b % a == 0) //自身就是答案  
        输出自身即可;  
  
    else{  
  
        printf("%d/%d=", a, b);  
  
        int c;  
  
        while(a != 1) { //输出后续结果
```

```

        c = b/a + 1; //商 + 1 作为 c

        printf("1/%d+", c);

        //只顾眼前（有前一个数对现在数的体现），贪心法的体现

        a = a*c - b; //更新 a

        b = b*c; //更新 b

        if(b % a == 0){ //b、a 可以整除

                printf("1/%d", b/a);

                break; //结束循环

        }

    }

}

}

}

```

源代码:

```

#include <stdio.h>

int main(int argc, char *argv[]) {
    int a, b;
    scanf("%d %d", &a, &b);
    if(a >= b)
        printf("不是真分数!");
    else if(a == 1 || b % a == 0) //自身就是答案
        printf("%d/%d=1/%d", a, b, b/a);
    else{
        printf("%d/%d=", a, b);
        int c;
        while(a != 1){ //输出后续结果

```

<pre>c = b/a + 1; //商 + 1 作为 c printf("1/%d+", c); //只顾眼前（有前一个数对现在数的体现），贪心法的体现 a = a*c - b; //更新 a b = b*c; //更新 b if(b % a == 0){ //b、a 可以整除 printf("1/%d", b/a); break; //结束循环 } } }</pre>	
运行结果： 3 5 3/5=1/2+1/10	
时间复杂度分析： $O(n^{1/2})$ 空间复杂度分析： $O(1)$	

二、实验总结

1. 深刻体会到了贪心策略的内涵，对于实际问题，要巧妙运用贪心，可以减少很多种情况的遍历，从而极大的提升时间效率
2. 掌握了利用贪心法解决实际问题的能力，“内功”得到了提升。
3. 对于成功建模的实际问题，证明了贪心可行之后，相比枚举、递归、回溯等方法效率极高，要巧妙运用。